FedBNR: A Fully Global Federated Gaussian Process

Haolin Yu, Kaiyang Guo, Mahdi Karami, Guojun Zhang, Hongliang Li, Xi Chen, and Pascal Poupart

Abstract. Uncertainty estimation plays a key role in many practical areas such as simulation and parameter optimization. Gaussian process (GP) is one popular model that provides natu- rally well-calibrated uncertainty estimates. However, it is challenging to learn a global GP posterior under the federated learning (FL) framework. In FL, clients' private data should not be shared, but merging local kernels directly leads to privacy leakage. Previous works that consider federated GPs avoid it and focus on the personalized setting. This sacrifices information from other clients that can be exploited to benefit generalization. We present Federated Bayesian Neural Regression (FedBNR) that learns a global federated GP while respecting clients' privacy. We incorporate deep kernel learning and random features by defining a unifying random kernel (URK). URK enables a principled approach of learning a global posterior as if all client data is centralized. Experiments conducted on real world regression datasets show statistically significant improvements.

Keywords: Federated learning · Gaussian process · Random feature · Bayesian method

1 Introduction

Uncertainty estimation has been an indispensable part of many practical areas. For example, in simulations widely used for the development of aircraft, trains, and vehicles (Danquah uncertainty estimation is needed to assess the uncertainty inherent in any simulation model (Riedmaier et al., 2021). Researchers have also been using Bayesian optimization to speed up material design with nanoparticles (Xia et al., 2023; Iyer et al., 2020, 2019), where uncertainty estimation is used to balance exploitation and exploration. In machine-assisted decision making (Begoli et al., 2019), it provides the trust needed to inform decisions. Lab. Gaussian process (GP) has been an especially popular model that provides uncertain estimates (Verrelst et al., 2013; Li et al., 2021). GPs with deep kernel learning (Wilson et al., 2016a, 2016b) provide a good balance between expressiveness and simplicity to represent model uncertainty. At one end of the spectrum, most traditional models such as traditional neural networks do not capture any uncertainty, but are simple and scalable. At the other end of the spectrum, Bayesian neural networks express a full distribution over all weights of neural networks, but exact inference tends to be infeasible. In between, a GP with a deep kernel consists of a deep neural feature extractor with a distribution

over the last layer that facilitates exact inference. Since the last layer is the most important for prediction, representing its uncertainty is often sufficient to capture most of the uncertainty of a model (Gawlikowski et al., 2022; Kristiadi et al., 2020). However, extending GPs to the federated learning (FL) setting itself is challenging. In FL, we seek to train a model in a distributed way across several clients without any data leav- ing the clients to preserve privacy. Several works have explored distributed GPs (Deisenroth & Ng, 2015; Yin et al., 2020) and federated GPs (Achituve et al., 2021; Kontoudis & Stil- well, 2023, 2024). While distributed GPs are designed to improve scalability, they pose an important privacy risk since sharing kernels either implies sharing data or sharing pairwise data similarity. In contrast, federated GPs including pFedGP (Achituve et al., 2021),DEC- apx-GP (Kontoudis & Stilwell, 2023), and DEC-gapx-GP (Kontoudis & Stilwell, 2024) share only the hyperparameters of kernels while learning local GP posteriors that are never shared. This personalized approach reduces privacy risks, but the local GPs do not benefit from other client information beyond the shared kernel hyperparameters. Thus, its accu- racy suffers when generalization is desired or new clients come without training data. For example, a hospital that only collected chest X-rays of lung emphysema patients may also want their model to help diagnose rib fractures with the same type of input. However, per- sonalization would prevent the model from generalizing to other ranges. Moreover, new clients with few training data cannot effectively train predictors with local GPs, even if they do not need generalization. To address such challenges, we propose a new federated GP technique called Federated Bayesian Neural Regression (FedBNR) that learns a global federated GP. We first define a unifying random kernel (URK) which allows random feature approximation of both sta- tionary and non-stationary kernels. FedBNR with URK avoids kernel sharing by working directly in the feature space and sharing scatter matrices, reducing privacy risks and decreas- ing the computational complexity to linear in the amount of data. Since FedBNR maintains a global posterior, it makes full use of different clients' information, and performs better for generalization and new clients. The approach is demonstrated on real world regression datasets where we achieve statistically significant improvements over prior techniques both in terms of point and uncertainty estimation. We further extend our algorithm with differential privacy to provide formal privacy guarantees. The main contributions are: • New random feature approximations of non-stationary kernels enabled by URK, and their concentration guarantees. • A new federated GP technique FedBNR, which achieves better generalization perfor- mance in both accuracy and uncertainty calibration due to its global posterior. The paper is structured as follows. In Sect. 2, we review essential background knowledge of GP, FL, and related works. In Sect. 3, we introduce URK and its theoretical guarantees, pro- pose FedBNR based on URK, and extend FedBNR with differential privacy (DP). Sect. 4 details synthetic and real-world experiments. Finally, Sect. 5 concludes the work.

2 Background

Notation. We use the following notation throughout the paper. X,X′ Rp × n, x,x′ Rp ∈ ∈ are input matrices or vectors. n is the number of data points and p is the number of features. y Rn is the target vector. x ,y indicate vectors that have not been seen by the models. σ ∈ R+ is the standard devia ∗ tion ∗ of the additive Gaussian noise in the models we men- ∈ tion later. I is the identity matrix and 0 is the zero vector. Their dimensions can be inferred from the context. ϕ:Rp Rd denotes a d dimensional basis function, and Φ=ϕ(X) → denotes the corresponding features of X. k:Rp Rp R denotes a kernel function, and × → K=k(X,X) is the corresponding kernel Gram matrix. w Rd are the weights of the linear ∈ layer in a Bayesian linear regression model. λ R+ is the prior standard deviation on w. ∈ E[],Cov(,), and Tr() are the expectation, covariance, and trace function. Any symbol · · · · with a c subscript is a local version of the original symbol, held by client c.

2.1 Gaussian Process and Random Fourier Feature

A GP can be viewed as an infinite dimensional Gaussian distribution over function objects f(). If restricted to a finite set of points of interest X on the support, a GP becomes a · multi-dimensional Gaussian distribution f(X), providing distributional information at these places. Formally, GP regression with additive Gaussian noise is established as y=f(x)+ϵ, where ϵ (0,σ2). After a prior over f() is specified, the likelihood, ∼N · posterior, and prediction can be computed as follows: Prior: f() (0,k(,)) (1) · ∼GP · · Likelihood: (yX,f()) (f(X),σ2I) (2) | · ∼N Posterior: (f()X,y) (m̄(),k′(,)), where (3) · | ∼GP · · · m̄()=k(,X)(K+σ2I)− 1y, (4) · · k′(,)=k(,) k(,X)(K+σ2I)− 1k(X,) (5) · · · · − · · Prediction: (y x ,X,y) (m̄(x),σ2+k′(x ,x)) (6) ∗| ∗ ∼N ∗ ∗ ∗ The runtime complexity of GP is O(n3) due to K−^{1}. Thus in practice, full GPs are often infeasible, and approximations are needed for scalability. The performance of GP models highly depends on the kernel function k(,), and hyperparameters of this function can be · · learned by maximizing logPr GP (y | X)= y⊤(K+σ2I)− 1y log | K+σ2I | . A valid kernel function is any positive definite function and can always be decomposed into the inner product of some basis functions k(x,x′)=ϕ(x)⊤ ϕ(x′) (Cuturi, 2009). An important advantage of working in the function space (i.e., with kernels) is that kernel func- tions can correspond to infinite features (i.e., the corresponding ϕ(x) must map x to infi- nitely many linearly independent features) without paying a price in complexity. Popular kernels, such as the Gaussian kernel k(x,x′)=exp(x x′ 2/2γ2) (γ R+ is a kernel −|| − || ∈ hyperparameter), tend to induce infinite dimensional features, providing significant model capacity. If the kernel has finite features (i.e. ϕ(x) that maps x into finitely many features), a GP ∃ degenerates into Bayesian linear regression (BLR) in the feature space with parametrized model f()=w⊤ ϕ(). Given a prior over w, the likelihood, posterior, and prediction are: · · Prior: w (0,λ2I) (7) ∼N Likelihood: (yX,w) (w⊤Φ,σ2I) (8) | ∼N Posterior: (wX,y) (w̄,A−^{1}), where (9) | ∼N w̄ =σ− 2A−^{1}Φy and A=σ−^{2}$\Phi\Phi$⊤+λ− 2I (10) Prediction: (y x ,X,y) (ϕ(x)w̄,σ2+ϕ(x

)⊤A−1ϕ(x)) (11) ∗| ∗ ∼N ∗ ∗ ∗ Recall ϕ(X) Rd. Then the complexity of BLR is O(nd2+d3) due ∈ to A−1. Hyperparameters of ϕ can be learned similarly by maximizing logPr BLR (y | X)= − nlogσ2 − log | λ2I |− log | A |−|| y − Φw¯ || 2 − λ − 2w¯⊤ w¯. It is very important to note that Eqs. 6 and 11 are identical, with only the order of matrix multiplication changed, resulting in different complexity and model interpretation. Random Fourier features (RFF) (Rahimi & Recht, 2007) can be used to approximate infinite stationary kernels with finite features based on Bochner's theorem: Theorem 2.1 (Bochner's theorem, e.g. (Rudin, 2017)). A continuous stationary kernel k(x,x′)=k˜(x x′)=k˜(δ) on Rp is positive definite if and only if k˜(δ) is the Fourier transform of a non-negative measure. Above, δ=x x′ and k˜ is the kernel k redefined in terms of one argument that is the difference of the inputs x x′. If the kernel is real-valued and properly scaled, its inverse Fourier transform p(ω) is also real-valued and is a proper probability distribution. Then a valid feature mapping is zω (x)=(cos(ω ⊤x),sin(ω ⊤x)), since k˜(δ)= ˆ p(ω)cos(ω⊤δ)dω=Eω [cos(ω⊤δ)]=Eω [zω (x)⊤zω (x′)] Rp The true expectation is then approximated by the empirical mean of zω (x)⊤zω (x′) with multiple samples from p(ω), which enables recovering an infinite kernel with a finite set of features and working directly in the feature space.

2.2 Federated Learning v.s. Distributed Learning

In federated learning (FL), there is always a collection of clients c S that wish to col- ∈ laborate, and each client holds their own data D c =(Xc ,y c) locally. We seek to train some machine learning model θ on these client data, with parameters θ. Conventionally, we M would centralize all the data D= c S D c, and learn the model globally. This simple approach becomes infeasible if the clie∈nts cannot directly share their data due to privacy ∪ concerns. FL is designed to specifically tackle this problem. In typical scenarios, a trusted central server is allowed to receive and send perturbed matrices that only contain limited information about raw client data such as model parameters. Although FL seems similar to distributed learning (DL) where tasks are also distributed to client workers to exploit extra computational resources, FL focuses on settings that are very different from typical DL settings and thus faces unique challenges that DL algorithms cannot handle. First, DL algorithms are usually deployed on clusters, where communication between nodes are fast and reliable, thus algorithms with huge message throughput, like aggregating local gradients at each iterative step, are practical and feasible. However, in the cross-device setting of FL, client nodes are often millions of mobile devices that are not reliable and possess only limited computational resources. In this case, it is crucial to reduce the communication cost, so FL algorithms are often expected to aggregate local models or information that have evolved for more than one step or epoch of gradient descents from the previous global consensus, and a line of works (Konečny et al., 2016; Sattler et al., 2019; Reisizadeh et al., 2020) are dedicated in improving the communication efficiency. This nature of clients further leads to data and system heterogeneity, since the server often, if not always, has no control over how

data or computation resource is distributed. More importantly, FL algorithms focus on reducing privacy risks of clients, while it is not a con- cern at all of DL algorithms. Hence, many DL algorithms send risky messages with which other entities can easily retrieve the raw data of clients. A review with more details can be found in Appendix A.

2.3 Related Work

Distributed and Federated GPs Closely related to our work is the literature on distributed and federated GPs. Distributed GPs (Deisenroth & Ng, 2015; Yin et al., 2020) were initially proposed to improve scalability by partitioning the data into several machines since GPs that operate in the dual space scale cubically with the amount of data in the worst case. The product-of-experts framework has emerged as a popular technique to aggregate local GPs, including generalized product of experts (Cao & Fleet, 2014) and robust Bayesian committee machines (Deisenroth & Ng, 2015). Distributed optimization of hyperparam- eters in GPs has also been explored using ADMM (Xie et al., 2019), statistical parameter matching (Yurochkin et al., 2019), or variational inference (Gal et al., 2014; Hoang et al., 2016). While those techniques do not ensure data privacy, recent work about federated GPs reduce privacy risks while training in a distributed way. This includes pFedGP (Achituve rameters of a global kernel using either FedAvg or decentralized proximal ADMM. DEC- gapx-GP (Kontoudis & Stilwell, 2024) is an extension of DEC-apx-GP, which shares an additional dataset with all clients as extra training data. Note this violates the constraints of FL, in the absence of a public dataset, since the additional dataset must consist of client's local data. These federated GP algorithms share only the hyperparameters of kernels, but learn local GP posteriors that are never shared. In another line of work, GPs have also been used to estimate correlations between clients in FL in order to actively select independent clients for aggregation (Tang et al., 2021). Our work differs from previous distributed and federated GPs by learning a global federated GP with better generalization performance. Deep kernel learning and GP approximations There have been many works that commit- ted to increasing the model capacity of GPs by incorporating deep neural networks (DNNs). Works of Hinton and Salakhutdinov (2007), Calandra et al. (2016) either pretrain a deep belief network or directly train it with a GP to extract first-step features before sending the data into the GP with conventional kernels. Wilson et al. (2016a, 2016b) build their algo- rithm on top of Wilson and Nickisch (2015), Wilson et al. (2015), Titsias (2009), Hensman and stochastic variational inference for classification tasks. Then Tran et al. (2019) studies the variance estimations of deep kernel learning models and proposes to use Monte-Carlo Dropout (Gal & Ghahramani, 2016) for better calibration. Ober et al. (2021) proposes to use Bayesian Neural Networks instead of deterministic DNNs to prevent over-fitting. Besides, Garnelo et al. (2018) designed a special architecture that makes it possible for DNNs to simulate GP behaviors. Damianou and Lawrence (2013) form GPs into a deep architecture that corresponds to a deep belief network based on GP mappings. To make GPs practi- cal, one

popular method is the inducing point approximation, where the joint GP prior of training points and inducing points are approximated (Quinonero-Candela & Rasmussen, 2005). Variants includes SoD (Quinonero-Candela & Rasmussen, 2005), SoR (Smola & Bartlett, 2000), DTC (Seeger et al., 2003), FITC (Snelson & Ghahramani, 2005), and PITC (Schwaighofer & Tresp, 2002). Later, KISS-GP (Wilson & Nickisch, 2015) gave another interpretation that inducing point approximations are equivalent to global GP interpolation, and it can exploit Kronecker structures (Saatçi, 2012). Another approximation method is random features (Rahimi & Recht, 2007; Sinha & Duchi, 2016; Oliva et al., 2016) that use randomized basis functions to approximate kernels. More information about scalable GPs can be found in this survey (Liu et al., 2020).

3 Federated Learning of Global GPs

Personalized federated learning (PFL) algorithms can learn fine-tuned local models for each client and can automatically handle preference distribution skew (Zhu et al., 2021) (i.e., Pr(yX) varies with clients). However, this comes at the cost of the generalization ability, | especially when preference skew is not a concern and the clients want out-of-distribution generalization w.r.t. its training data. If a client is not capable of collecting enough data in the same range as their desired prediction, the accuracy of PFL local models could suf- fer. For example, suppose the posterior of a GP with a stationary kernel is conditioned on local training data (x1 , $\cdots$,xnc) $\subset$ Bc, an infinite compact set, and (y 1 , $\cdots$,y nc). Due to non-i.i.d. clients, it is common that other clients c $'$ have a different Pr(X) in the form that Bc $\cap$ Bc$'$ =Bc$'$. That is, training data of client c $'$ is considered out-of-range for client c. Suppose client c wants to generalize to the range of all clients' training data c SBc. The $\cup \in$ prediction of data points from other clients' range x$'$ / Bc will gradually become the con- $\in$ stant zero prior as x$'$ moves further from Bc, regardless of what y $'$ really is. Thus, this gener- alization error is unbounded. However, a global GP does exhibit a uniformly bounded error (Lederer et al., 2019), since its posterior is conditioned on all clients' data. It means all the clients, whether or not possessing training data, can benefit from others' information in FL. With such motivation for learning global federated GPs, we derive our algorithm in the following sections. In Sect. 3.1, we propose a random feature formulation URK, which is more flexible and compatible to deep learning as it can recover non-stationary kernels. It also provides more model capacity and thus better accuracy compared to plain deep learning without random features. Then, we detail our algorithm FedBNR in Sect. 3.2, showing how a federated global posterior can be learned with URK. Finally, we extend FedBNR with DP in Sect. 3.3, providing formal privacy guarantees (Fig. 1).

3.1 GP Under Random Feature Maps

URK Definition Recall that a kernel function can be rewritten as the inner product of some basis functions k(x,x$'$)=ϕ(x)$\top$ ϕ(x$'$). We propose the following kernel

and random fea- ture approximation. Let ω Rd′ be any random variable or vector, and g:Rd′ Rp Rd be any function $\in \times - \rightarrow$ that extracts d features out of each input with random weights ω. We construct the random basis functions z as zω (x)=g(ω,x). We define the true underlying kernel and its approxi- mation, the unifying random kernel (URK) as: k(x,x′)=Eω [zω (x)⊤zω (x′)] (12) ≈ URKm ω,g (x,x′):= ($\sqrt{}$ s m m ω (x)) ⊤ ($\sqrt{}$ sm ω m (x ′)) =ϕ(x)⊤ϕ(x′) (13) Fig. 1 FedBNR learns a global federated GP in two phases: kernel learning with federated optimization and last layer updating with exact Bayesian inference where sm ω (x) is the concatenation of m samples of zω (x). Equation 12 is the true underlying kernel we construct. Equation 13 is the Monte-Carlo approximation of Eq. 12, where the samples are the random features that URK produces. Furthermore, note Eω [zω (x)⊤zω (x′)]=Tr(Covω (zω (x),zω (x′)))+Eω [zω (x)]⊤Eω [zω (x′)] (14) Equation 14 follows from the definition of covariance (i.e., Cov(A,B)=E[AB] E[A]E[B], where A, B are random variables). Tr refers to the trace, which is needed for random vectors. It is a decomposition of the true underlying kernel, which provides an intuitive understand- ing of the construction where URK mainly captures the covariance between different ran- dom basis functions. This aligns with the purpose of the kernel function. Examples and Theoretical Guarantees Since g is an arbitrary function, we can assign it any DNN with any architecture. We call it a kernel neural network (KNN) since its weights, θ, are essentially the kernel hyperparameters. It is trivial to map a deep kernel defined by k(ϕ θ (x),ϕ θ (x′)), where k is any conventional stationary kernel and ϕ θ is any feature map- ping function potentially represented by a deep neural network, into a URK construction: ω ∼ p(ω),g(ω,x)= sin(ω⊤ϕ θ (x)),cos(ω⊤ϕ θ (x)) ⊤ (15) [ ] where p(ω) is the inverse Fourier transform of k. With m samples of ω, the error of approxi- mating the true underlying kernel k(x,x′) drops exponentially: Theorem 3.1 x,x′ Rp, p(ω) being a proper distribution over Rd′, and ∀ ∈ ∀ g :Rd′ × Rp − → Rd has the form of Eq. 15. With m ∈ Z+ samples of ω, ∀ t >0 mt2 Pr[| URKm ω,g (x,x′) k(x,x′) |≥ t] ≤ 2exp −8 ( ) URK can reconstruct not only conventional stationary kernels with a RFF mapping, but also non-stationary kernels beyond RFF's capability. We give two example constructions and their corresponding convergence properties below. Full proofs are in Appendix D. The first example recovers a non-stationary kernel that induces infinite basis functions, like the Gaussian kernel. P U r R op K o m ω si , t g i (o x n , x′ 3) . i 2 s a L n e o t n-s ω tat ∼ ion N ar (y 0 , k I e) r , n g e (l ω w , it x h) i = nfin e i x te p (fe ω a ⊤ tu x r) e , s . then k(x,x′)=lim m − → ∞ The following theorem bounds the error of URK approximation for the construction in Prop. 3.2. Fenton-Wilkinson approximation (Fenton, 1960) is used to model the summation of log-normal distributions, and the error is bounded by the sum of two tail probabilities. Theorem 3.3 Let ω and g have the form of Prop. 3.2. Suppose x,x′ Rp satisfies ∈ || x || 2 ≤ a, || x′ || 2 ≤ a′. With m ∈ Z+ samples of ω, ∀ t ∈ (0,exp(a+a′)/2)), by Fenton-Wilkinson approximation, exp(a+a ′) 1 σ s :=log m − +1 ,E :=exp((a+a′)/2) ( ) Pr[| URKm ω,g (x,x′) k(x,x′) |≥ t] ≤ F 2log(t+E) σ (a+a ′)+σ s (s) +F 2log(E t) (a+a ′)+σ s 2σ (s) F is the cumulative probability distribution

function of the standard normal distribution. The second example recovers a popular non-stationary kernel, the polynomial kernel. Proposition 3.4 Let c̃,ñ R be hyperparameters of the polynomial kernel ∈ k(x,x′)=(x⊤x′+c̃)ñ. Define p̃=[1 , p , p , · · · , p]⊤ ∈ Rp+1, ω ∼ Multi(ñ,p̃), the multinomial distribution. Define x̃=[√2c̃, √2px⊤]⊤ Rp+1, g(ω,x)=exp(ω ⊤logx̃), ∈ then k(x,x′)=lim m URKm ω,g (x,x′)=(x⊤x′+c̃)ñ. − → ∞ The following theorem bounds the error of URK approximation for the construction in Prop. 3.4, with Hoeffding's inequality (Hoeffding, 1994) similarly to Thm. 3.1. Theorem 3.5 Let ω and g have the form of Prop. 3.4. Suppose entries of x,x′ ∈ R p + are bounded by [a, b]. With m ∈ Z+ samples of ω, ∀ t >0, max √2c̃,√2pb v:= { } min √2c̃,√2pa { } 2mt2 Pr[| URKm ω,g (x,x′) k(x,x′) |≥ t] ≤ 2exp − v4ñ( ) Without Randomness Note the deep features themselves already allow a valid BLR for- mulation that is equivalent to a linear kernel. However, it is restricted to finite basis func- tions as the same size of the second last layer and has the implicit assumption that the target is linear in the features. As the most basic kernel function, the linear kernel does not provide additional model capacity to the deep neural network. Previously, deep kernel learning algo- rithms (Wilson et al., 2016a, 2016b) apply the Gaussian kernel, which induces infinite basis functions, to deep features and show they effectively improve the model accuracy compared to using deep features only. The rationale is straightforward: adding extra basis functions to a model never worsens its training accuracy, and GPs are resilient to over-fitting. Adding randomness with the URK construction follows the same idea. It further extends the model capacity of the KNN by allowing an effective approximation to infinite kernels. In Appen- dix C.1, we report an extra ablation study that shows empirically the URK approximation indeed improves the accuracy in most cases compared to deep features only. Learn the Distribution URK is meant to be a generic definition that enables any archi- tecture g and any distribution for ω. Whether a specific instantiation is preferable is highly problem dependent. Nevertheless, we present an architecture that utilizes the flexibility of deep learning and let the model learn automatically a distribution for ω that best fits its data. The architecture is derived in alignment with deep kernel learning proposed by Wilson et al. (2016b). As illustrated in the left half of Fig. 2, URK can recover any DNN combined with a conventional stationary kernel, using the architecture mentioned. p(ω) of popular station- ary kernels are reported in RFF (Rahimi & Recht, 2007). For example, the Gaussian kernel simply corresponds to ω (0,I), the standard normal distribution. Furthermore, we can ∼N exploit the flexibility of URK and define a distribution shifter h that transforms ω. We can start from a standard normal distribution, which is very easy to sample from, and send the samples through h to simulate a much more complex distribution with minimal computa- tional resources required. We choose h such that the identity function is in its hypothesis set, which can be achieved by adding (potentially linearly transformed) ω to the output of any architecture, similarly to the residual network (He et al., 2016). Then, we can learn a distri- bution for ω that is at least as good as the Gaussian kernel in deep kernel learning.

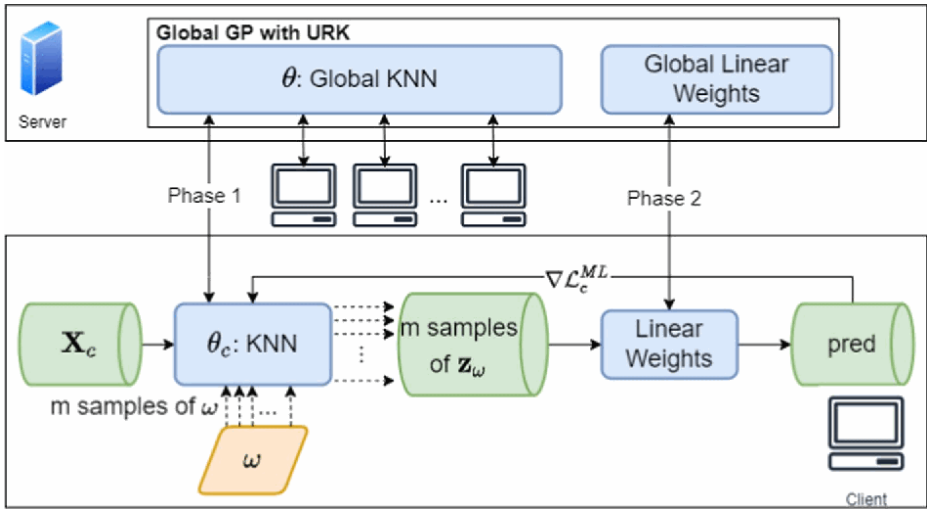

Fig. 1. FedBNR learns a global federated GP in two phases: kernel learning with federated optimization

3.2 FedBNR: Federated Learning of Global GPs with URK

With URK, we can now approximate infinite deep kernels with finite random features and work in the feature space. The training procedure of our proposed algorithm FedBNR can be divided into two phases, as illustrated in Fig. 1. In the first phase, we train the KNN by optimization methods. In the second phase, we calculate the weights of the last linear layer that maps the random features to the output space, by a closed-form formula inferred from Eq. 11. The pseudocode of FedBNR is summarized in Algorithm 1. In phase 1, we follow a standard training procedure under the FL framework. We assume there is a central server holding the shared global KNN weights θ and global hyperparam- eters σ,λ that denote the noise level and the prior variance respectively. We assume there is a set of clients c S holding local KNN weights θ c, local hyperparameters σ c ,λ c, local $\in$ inputs Xc, and local targets y c . We use ϕ θ (ω,x) to denote the resulting features of x with a specific set of random samples ω with respect to θ. ω is omitted for simplicity when it is not fixed. In the beginning of each aggregation round, the server sends a copy of aggregated or initialized θ,σ,λ to all the clients. All the clients c S first update their local model for $\in$ Fig. 2 From left to right: a standard deep kernel learning algorithm with conventional stationary kernel GPs; the corresponding URK architecture; a more general architecture enabled by URK for convenient latent stationary kernel learning a fixed number of iterations. Then they send θ c, σ c, and λ c back to the server for aggrega- tion and start another aggregation round. The local loss function is the negative local log marginal likelihood: L M c L = logPrBLR (y c| ϕ θc (Xc);σ c ,λ c) (16) =n c logσ c 2+log(| λ2 c I || Ac |) || y c− Φ c w¯c || 2+λ− 2w¯c⊤w¯c . One commonly used method for aggregation is the FedAvg (McMahan et al., 2017) heu- ristic,

where the new global model parameters are assigned the average of all client model parameters θ= c S θ c / | S | . However, multiple works (Karimireddy et al., 2020; Shoham $\sum$ heuristic can suffer from non-i.i.d. clients. To account for this, we propose to adapt kernel knowledge distillation (He & Ozay, 2022) to aggregate the KNNs. We assume the server holds a relatively small dataset Xkd ,y kd and tries to minimize the following knowledge distillation loss with respect to this dataset: L KD = L M kd L+α $*$ MSE(ϕ θ (Xkd)$\top\phi$ θ (Xkd), ϕ θc (Xkd)$\top\phi$ θc (Xkd)/ | S |). (17) c S $\sum$ $\in$ here, ML is the log marginal likelihood loss ML with respect to the knowledge distilla- Lkd L tion dataset Xkd ,y kd . α is a common hyperparameter in knowledge distillation methods to adjust the ratio between the log marginal likelihood loss and the mean squared error (MSE) loss. The MSE loss forces the global kernel ϕ θ (Xkd)$\top$ ϕ θ (Xkd) to simulate the mean of all client kernels, namely c S ϕ θc (Xkd)$\top$ ϕ θc (Xkd)/ | S | , which is akin to concatenating all $\in$ the features of the client kernels. Ideally, if the global kernel successfully learns to do so, it $\sum$ should not perform worse on any of the clients, while the FedAvg heuristic has no similar guarantees. In phase 2, we fix the kernel hyperparameters and learn w¯ in an exact way as if all client data is centralized. First, note that we can decompose w¯ as follows: w¯ =$\sigma-^2(\sigma-^2\Phi\Phi\top+\lambda-$ 2I)$-^1\Phi$y (18) c S c S $\sum$ $\in$ $\sum$ $\in$ Φ c =ϕ θ (Xc) denotes the random features of local inputs extracted by the global KNN. The server first broadcasts the global model to all the clients and asks them to return the scatter matrices Φ c $\Phi\top$c and the vectors Φ cy c . Then, the server simply takes the summation of cli- ent messages, calculates w¯ according to Eq. 19, and broadcasts the whole model again to all the clients. Note that phase 2 only needs to be done once after kernel learning to achieve the global model. There is no need to iterate between the two phases. In phase 1, each local update costs O(n c (md)2+(md)3) time to compute and inverse Ac and O((md)2) space to store Ac. This is equivalent to the complexity of BLR models with random feature approximations. It is more scalable than working with the kernel K directly, which costs O(n3 c) time and O(n2 c) space when md $\ll$ n c. Each client shares a message of size θ with the server. FedAvg aggregation then costs O(θ) time and O(θ) space, while | | | | | | KD aggregation costs O(n kd (md)2+(md)3+n2 kd md) time and O((md)2+n2 kd) space, where n kd is the size of the knowledge distillation dataset. In phase 2, each client calculates the scatter matrices and the vectors, costing O(n c (md)2) time and O((md)2) space. Each client shares a message of size (md)2 with the server. Finally, learning the global posterior costs O((md)3) time and O((md)2) space, similarly to phase 1. Note that if n id inducing points are used to approximate the full kernel (e.g. in pFedGP), O(n3) time and O(n2) id id space is needed similarly. Algorithm 1 Federated Bayesian neural regression (θ: the global KNN, σ: noise level, λ: prior covariance of the linear layer, ω: a set of random numbers, ζ: local learning step, ζ': knowledge distillation step)

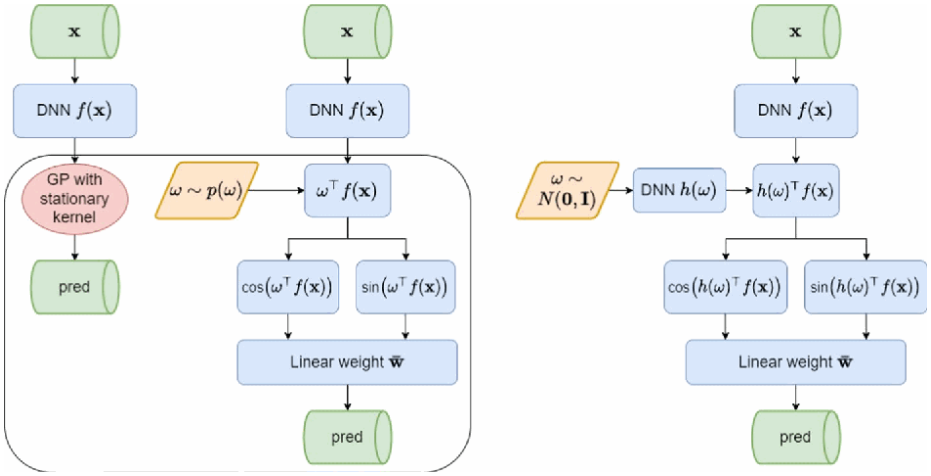

Fig. 2. From left to right: a standard deep kernel learning algorithm with conventional stationary kernel

3.3 Enhance Protection with DP

We claim that FedBNR protects the privacy of clients at least as well as FedAvg and other FL algorithms that send client models to the server. In phase 1, the aggregation only requires client model parameters. In phase 2, we send two messages outside each client: Φ c $\Phi\top$c and Φ cy c . These matrices and vectors (of size m $\times$ m and m) are also highly compressed summaries of the raw data, which prevents immediate privacy invasions as discussed in Appendix A. We note however that FL with compressed messages does not prevent privacy attacks (Boenisch et al., 2021; Mothukuri et al., 2021), it simply makes it harder to infer the underlying data. To provide formal privacy guarantees, federated learning needs to be com- bined with formal mechanisms such as differential privacy (DP) (Wei et al., 2020; Triastcyn & Faltings, 2019) or secure multi-party computation (MPC) (Truex et al., 2019; Byrd & Polychroniadou, 2020; Li et al., 2020). Definition 1 A mechanism M satisfies (ϵ, δ)-DP if for any two datasets D,D $'$ that differ by only one record (i.e. (D D $'$) (D $'$ D) =1, noted as D D $'$), and for any possible $| - \cup - |$ $\sim$ output O Range(M), $\in$ Pr[M(D)=O] Pr log $>\epsilon$ $<\delta$. O $\sim$ M(D) [(Pr[M(D $'$)=O])] The idea of (ϵ,δ)-DP is to add enough noise to the shared information so that with high probability (0 δ<1), the distribution of the shared message does not change much when $\leq$ the raw dataset is modified by one record (controlled by ϵ>0). Thus, one cannot effectively tell if any record is included in the dataset. (ϵ,δ)-DP models the information leakage of the raw dataset with the inner log ratio of Def. 1, and guarantees that it is bounded by ϵ, the pri- vacy budget, with probability δ. Empirically, people set ϵ^1 for strong privacy protection, $\leq$ while ϵ 10 is deemed weak. We will use the following mechanism: $\geq$ Theorem 3.6 (e.g. (Kerkouche et al., 2021)). A Gaussian Mechanism is (ϵ,δ)-DP with GM information revealed f: GM (D)=f(D)+ (0,σ2I),

where σ=L (f) 2ln(1.25/δ)/ϵ. $\surd$ L (f) calculates the L sensitivity of function f as follows. L (f)=m D, D x ′ || f(D) f(D′) ||2 ,D ∼ D′. (20) We now extend FedBNR to be differentially private with the Gaussian mecha- nism. In phase 1, the L sensitivity of the kernel hyperparameters is the L gradi- ent clipping constant times the number of local update rounds times the learning rate ζ. In phase 2, we send [Φ c $\Phi\top$c ,Φ cy c ,y$\top$c y c] instead and its L 2 sensitivity is 32+16/m+8b2+b4, where b:=max(y) min(y), assuming the URK construction $\surd$ is g(ω,x)=[sin(ω $\top$ ϕ NN (x)),cos(ω $\top$ ϕ NN (x))]$\top$. See details in Appendix A. Then, notice the following matrix is PSD without DP: [Φ y$\top$c c] $\Phi\top$c y c = [(Φ Φ c c y Φ c $\top$c)$\top$ Φ y$\top$c c y y c c] . (21) [ ] After adding Gaussian noise, we project the matrix back to the nearest PSD matrix by set- ting its negative eigenvalues to 0. Then we perform regular BLR with projected Φ c $\Phi\top$c and Φ cy c .

3.4 Synthetic Experiments

We designed the following synthetic experiments to show local GP models cannot handle generalization or new clients. We first decide a true underlying function, sample 200 points uniformly from the range [5,5], and add Gaussian noise with σ=0.5. We then learn the kernel hyperparameters in a centralized fashion to eliminate any impact from imperfect ker- 2. We train pFedGP and FedBNR with the learned kernel hyperparameters fixed and query for prediction in range [5,5]. The top left graph of Fig. 3 shows the result of pFedGP, and the top right one shows the result of FedBNR. The blue curve shows the true underlying function, and the green curve shows the predictions. The light red area is a 95% confidence interval based on variance. The purple dots are the data leveraged by each model for the last layer. Since both algorithms learn hyperparameters centrally, the only difference lies in whether they leverage data of both clients for the predictive model. As shown in the pFedGP graph, the personalized model of client 1 only sees its own data, so it does not generalize well to the range of client 2, while the global model learned by FedBNR leverages all the data and generalizes to the full range. Next, we introduce two new clients into the system. The first client holds data uniformly kernel hyperparameters learned centrally and train pFedGP, FedLoc, and FedBNR on these new clients separately. For testing, we still query the range [5,5]. As shown in the bot- tom graphs of Fig. 3. The quality of prediction of the pFedGP model is massively impacted when there are few points in the first case, and for FedBNR the loss remains approximately a straight line. For FedLoc, the loss is also impacted due to zero-out effects of non-over- lapping client models. Even worse, when training data and testing data are not in the same range for the second new client, the MSE loss of both pFedGP and FedLoc never drops back to the level of previous clients. Fig. 3 Top graphs: (i) left: prediction of pFedGP trained on two non-overlapping clients; (ii) right: global prediction of FedBNR. Blue curve: underlying truth; green curve: point estimation; red range: 95% con- fidence interval; purple dots: training data leveraged by the model. Bottom graphs: MSE of predictions tested on the same

range v.s. number of training data from a new client; (i) left: client with training range

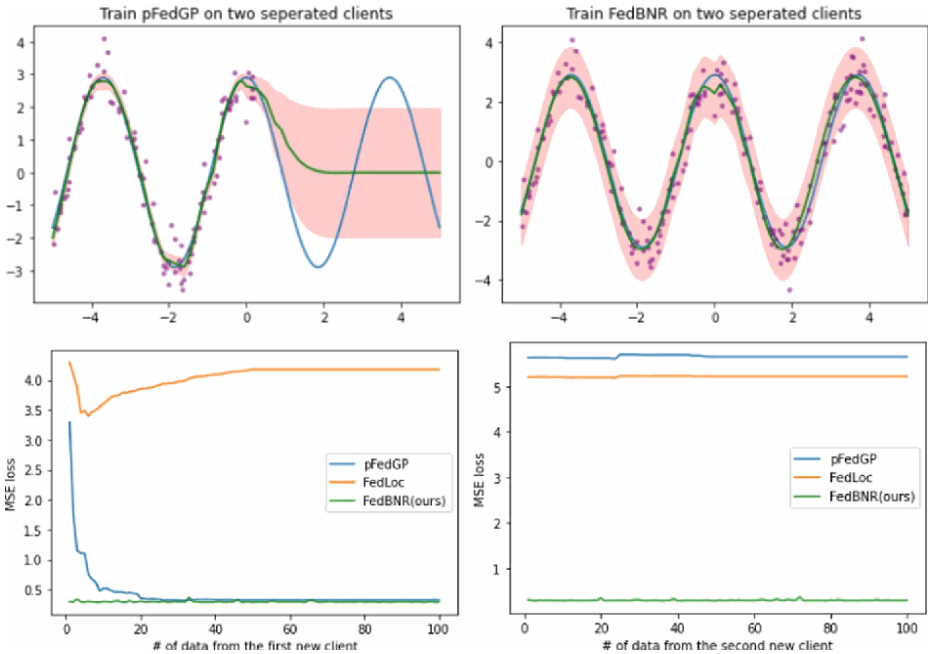

Fig. 3. Top graphs: (i) left: prediction of pFedGP trained on two non-overlapping clients; (ii) right: global

3.5 Real World Regression Tasks

To show our algorithm achieves better generalization performance in terms of both point and uncertainty estimation, we conducted comprehensive experiments on 5 real world regression tasks under 10 cases, including Skillcraft (Thompson et al., 2013), SML (Zamora-Martinez et al., 2014), Parkinsons (Tsanas et al., 2009), Bike (VE & Cho, 2020), and CCPP (Tüfekci, 2014). We note while a common challenge for GPs is to scale to large datasets, in FL, the challenge is the opposite: how to combine models trained on small datasets. When the datasets of each client become sufficiently large, then there is no need to combine models nor do FL. Hence, the experiments focus on reducing the amount of data per client while increasing the number of clients. The scalability of FedBNR is trivially guaranteed by the use of random features. Results of two variants are reported: (i) FedBNR that performs the FedAvg heuristic at aggregation; and (ii) FedBNR-KD that performs the knowledge distillation method at aggregation.

Two groups of baselines are compared to our method for RMSE error: (i) ablation study that includes local+local, local+global, avg+local, and kd+global, in the format of kernel learning method + last linear weight learning method, where we remove the global aggregation of either Phase 1 or Phase 2 from our methods; (ii) previous works that include (1) FedAvg (McMahan et al., 2017), a standard non-Bayesian FL algorithm that has a global model; (2) FedProx (Li et al., 2020), a non-Bayesian FL algorithm that adds a proximal loss to FedAvg to prevent client models from getting too far from the global model; (3) pFedGP (Achituve et al., 2021), a Bayesian PFL method that learns a local GP with a shared deep kernel for each client using FedAvg; (4) FedLoc (Yin al., 2019) without deep kernel learning; (5) DEC-apx-GP (Kontoudis & Stilwell, 2023), a decentralized Bayesian PFL method that learns a local GP with a shared separable squared exponential kernel for each client using proximal ADMM; and (6) DEC-gapx-GP (Kon- toudis & Stilwell, 2024), an extension of DEC-apx-GP that shares an additional dataset with all clients as extra training data. We also compare to pFedGP, FedLoc, DEC-apx, and DEC- gapx for calibration errors since they are Bayesian models that have a notion of confidence. Besides, we report results of a centralized GP equipped with URK as a casual reference of the testing error lower bound. We used fully connected neural networks to extract first- step features for FedAvg, FedProx, and pFedGP. We used Gaussian kernels for the GPs in pFedGP and FedLoc. For fairness, KNNs used by our methods have similar architectures to the combined kernel of pFedGP. FITC (Snelson & Ghahramani, 2005) approximations are implemented for pFedGP and FedLoc as described in pFedGP. We used 50 random samples for KNN and 50 inducing points for pFedGP and FedLoc. Further details are in Appendix B. Each dataset is uniformly divided into training, testing, and validation sets globally with a ratio of 8:1:1. The training data is sorted by the feature that has the largest absolute cor- relation coefficient with the output, and divided into multiple chunks. Each client randomly takes two chunks so their data distributions are heterogeneous. The larger the absolute cor- relation coefficient is, the more significant the distribution skew is. Before training, we tune hyperparameters that cannot be learned by gradient descent with grid searching on the validation set. Specially, FedBNR-KD uses 80% of the validation set for knowledge distillation, and the rest 20% for validation. All datasets are ran for 50 local epochs times at most 100 aggregation rounds in full batches. Validation and testing errors are recorded for each aggregation round. For methods that contain only local models, these errors are defined as the mean error of all local models with respect to the testing/validation set. If the validation error has not improved for 5 rounds, the training process is terminated. We report the average minimum testing RMSE for 10 random seeds for each case in statistical significance of the results compared to FedBNR with one-tailed Wilcoxon signed- rank tests (Wilcoxon, 1992). We report expected calibration errors (ECE) in Table 2 and per- form the Wilcoxon test compared to FedBNR-KD. ECE is a calibration metric that assesses the uncertainty estimation. Further metrics are included in Appendix C. The results show: (i) in terms of RMSE, our methods are statis-

tically better than the other models in most of the cases, especially when the training set at each client is significantly smaller than the whole set; (ii) compared with the ablation study methods, our methods always perform better, so the global aggregation at both phases is essential; (iii) FedLoc without deep kernel has a much smaller model capacity; (iv) FedBNR-KD only outperforms FedBNR in 40% of cases in terms of RMSE, which is probably due to the small size of data (8% of all) used for knowledge distillation. However, it is clearly more stable when client heterogeneity gets worse as the number of clients increases. It is also better calibrated in most cases.

Table 1. , which assesses the point estimation of different models. We also measured the statistical significance of the results compared to FedBNR with one-tailed Wilcoxon signed- rank tests (Wilcoxon, 1992). We report expected calibration errors (ECE) in Table 2 and per- form the Wilcoxon test compared to FedBNR-KD. ECE is a calibration metric that assesses the uncertainty estimation. Further metrics are included in Appendix C. The results show: (i) in terms of RMSE, our methods are statistically better than the other models in most of the cases, especially when the training set at each client is significantly smaller than the whole set; (ii) compared with the ablation study methods, our methods always perform better, so the global aggregation at both phases is essential; (iii) FedLoc without deep kernel has a much smaller model capacity; (iv) FedBNR-KD only outperforms FedBNR in 40% of cases in terms of RMSE, which is probably due to the small size of data (8% of all) used for knowledge distillation. However, it is clearly more stable when client heterogeneity gets worse as the number of clients increases. It is also better calibrated in most cases. 4.3 Personalization We conducted experiments with the personalization setting to show that the global GP posterior benefits uncertainty estimation even in the personalization setting. FedBNR can be naturally extended to the personalization setting by fine-tuning the global model.

During fine-tuning, we minimize L F c T $=$ **|| y c**$-$ ϕ θ**c (Xc)wc || 2 2** $+\lambda$ $-$**c 2 || wc || 2 2 , the local r**

3.6 Personalization

We conducted experiments with the personalization setting to show that the global GP posterior benefits uncertainty estimation even in the personalization setting. FedBNR can be naturally extended to the personalization setting by fine-tuning the global model. larized least square loss. Note the weights in the last linear layer are also updated by gradient descent. Varying the number of epochs of fine-tuning offers a controllable trade-off between generalization and personalization. Training datasets are distributed to clients as before for heterogeneity, but testing and validation sets are sampled with the same distribution as local pxADMM (Xie et al., 2019) is a distributed GP hyperparameter learning algorithm used by FedLoc and we extend it with deep kernels. The results show that FedBNR achieves compa- rable accuracy and better calibration than other methods, indicating a global posterior helps calibrate local models.

3.7 Differential Privacy

We conducted experiments for FedBNR with DP to show the trade-off between model utility and the privacy budget ϵ. Experiment details are similar to Sect. 4.2. The results of the Skillcraft and SML datasets are summarized in Fig. 4. As the graph shows, the RMSE loss of fully differentially private FedBNR quickly drops back to the RMSE of vanilla Fed- Log, when $\epsilon^0.1$. The DP noise affects the model less when there are more clients, since $\geq$ in Phase 1 the messages are averaged. Extra results are in Appendix C.

Table 2. Real world regression datasets, RMSE reported for the generalization setting. and denote sig- nificantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly Skillcraft SML Parkinsons Bike CCPP Train/test size 2670/334 3309/414 4700/587 7008/876 7654/957

corr-coef 0.660 0.783 0.410 0.539 0.948
#clients 10 100 10 100 10 100 10 100 10 100
Central GP 0.95 0.21 3.58 0.37 4.02
FedAvg 1.00↑ 1.03⇑ 0.36⇑ 0.71⇑ 6.83⇑ 7.38⇑ 0.38↓ 0.42 4.45 4.44
FedProx 0.98 1.05⇑ 0.34↑ 0.62⇑ 6.32⇑ 7.38⇑ 0.39 0.42 4.43 4.47
FedBNR 0.98 0.97 0.25 0.44 3.10 5.42 0.39 0.42 4.40 4.51

Table 3. ECE reported for the generalization setting. , , , and denote significantly worse or better results similarly to Table 1 Skillcraft SML Parkinsons Bike CCPP

#Clients 10 100 10 100 10 100 10 100 10 100
Central GP 0.02 0.09 0.27 0.11 0.24
FedBNR 0.05 0.20⇑ 0.39⇑ 0.37⇑ 0.36↑ 0.40⇑ 0.07 0.04⇑ 0.24⇓ 0.20⇓
FedBNR-KD 0.05 0.06 0.20 0.21 0.29 0.30 0.08 0.09 0.30 0.31

4 Conclusion

In this work, we proposed FedBNR, a novel algorithm that learns a global federated GP that provides better generalization and uncertainty estimation. FedBNR learns a deep kernel under feature mapping, and shares scatter matrices instead of direct features to achieve the exact global optimum of the last layer. It shows empirically statistically significant improve- ments in terms of point and uncertainty estimation compared to other federated GPs. One limitation of FedBNR is that the decomposition of global posterior learning with random features is restricted to regression tasks, and it is non-trivial to extend to the GP classification case where Laplacian approximation is needed. Fig. 4 FedBNR+DP results. Left: Skillcraft dataset; right: SML dataset. Phase 1 DP: add DP protection to θc only; full DP: add DP protection to all messages

Table 4. RMSE (top half) and ECE (bottom half) reported for the personalization setting. and denote significantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
pFedGP 1.00⇓ 1.08⇓ 0.31⇑ 0.64 2.97⇑ 3.66⇑ 0.47 0.44⇓ 4.72⇑ 12.66⇑
FedBNR 1.12 1.25 0.25 0.62 1.39 1.75 0.47 0.57 3.89 4.77
FedBNR-KD 1.12 1.22⇓ 0.38⇑ 0.66 1.78⇑ 2.13⇑ 0.49⇑ 0.63⇑ 3.95⇑ 4.74
10 100 10 100 10 100 10 100 10 100
FedBNR 0.01 0.04 0.24 0.28 0.04 0.04 0.11 0.08 0.25 0.33
FedBNR-KD 0.01 0.03⇓ 0.13⇓ 0.17⇓ 0.06⇑ 0.09⇑ 0.11 0.07 0.29⇑ 0.33
Fig. 4 FedBNR+DP results. Left: Skillcraft dataset; right: SML dataset. Phase 1 DP: add DP protection to

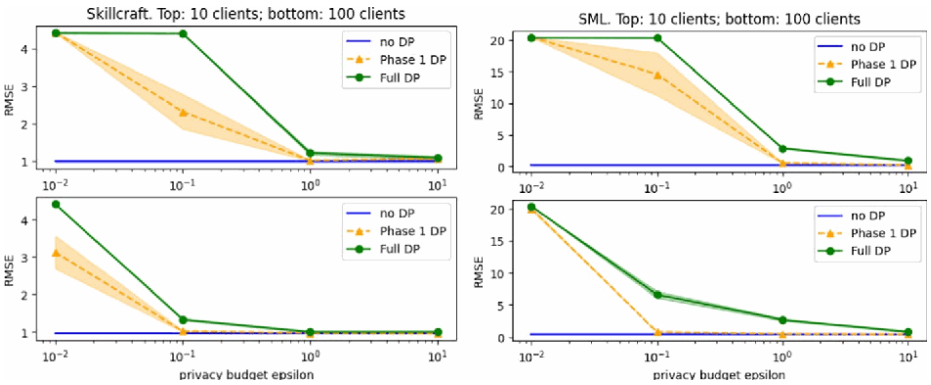

Fig. 4. FedBNR+DP results. Left: Skillcraft dataset; right: SML dataset. Phase 1 DP: add DP protection to

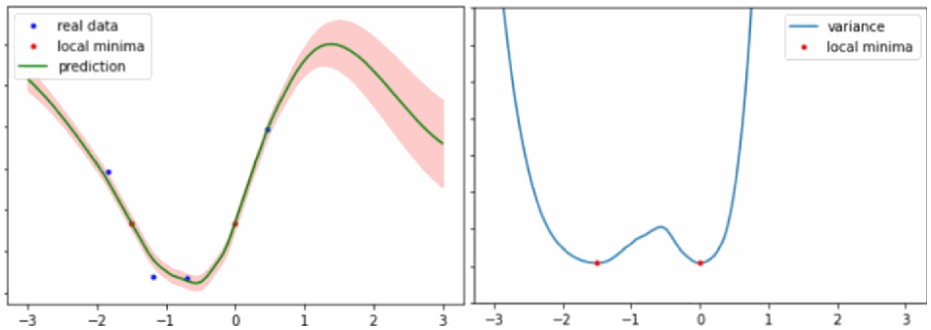

Fig. 5. An example to show why variance adversarial attack fails when we have a non-stationary kernel.

Table 5. Privacy leakage of dif- Technique Risky message Prob- Leaked ferent GP algorithms lematic info term Xc or Φc − − Xc, yc Inducing points U or Uc − − Xc ⊂ k(Xc,U) − − yc FedAvg (2017) N/A N/A N/A pFedGP (2021) N/A N/A N/A FedBNR (ours) N/A N/A N/A Centralized GP Xc/Φc K, Xc,yc k(x⋆,X) Risky message: messages that leak privacy but unavoidable FITC (2005) U, k(Xc,U) k(X,U) ⊂ Xc,yc for global prediction. pFedGP + Global Predictor U, k(Xc,U) k(X,U) ⊂ Xc,yc Problematic term: terms Distributed GP (2015) x⋆/Φ⋆ or k(x⋆,Xc)Xc,yc that the risky messages try Xc/Φc or to compute. Leaked info: X ,y private information that can be ∗ ∗ Distr. Variational GPs (2014; U k(U,) Xc recovered by adversaries with · ⊂ 2016) the risky messages. means "part of" ⊂ SPAHM GP (2019) U k(U, ·) ⊂ Xc with them. We then discuss privacy leakage of different GP algorithms and summarize them in Table 4. We will now discuss messages and algorithms listed in the table one by one. Risky Messages Features Xc or Φ c Without loss of generality, we can denote the prediction model by y=f(ϕ(x)), where ϕ(Xc)=Φ c. Then the adversaries can simply compute y c =f(Φ c) to recover the targets of clients. This further extends to sharing x⋆ or Φ ⋆, as the testing points and their corresponding targets can also be confidential. Inducing Points U or Uc There are different ways of choosing inducing points. One way is to select a subset of the training data X, and this clearly leaks raw data. Another way is to optimize them as hyperparameters. However, Smith et al. (2021) pointed out that inducing points optimized in this way often end up in regions of low data density and may corre- spond to outliers. In this case, there is a high risk that some inducing points will correspond directly to real data. They proposed to use differential privacy to protect the output, but the input is left unprotected. Even in the case where inducing points lie in different spaces than the original input, it is still possible to partially recover real y with the same argument as c Φ c. Thus we want to refrain from using inducing points. Cross Kernel Matrix k(Xc ,U) In general, sharing k(Xc ,U) does not reveal Xc imme- diately since the kernel function k tends to be non-invertible. However, in many algorithms that use inducing points, k(Xc ,U) serves as sufficient statistics to predict y c . For example, for FITC (Snelson & Ghahramani, 2005), the predictive mean is given by: f(x⋆)=k(x⋆ ,U)Σk(U,X)Λ− 1y, where Σ=(k(U,U)+k(U,X)Λ− 1k(X,U))− 1 Λ=diag[K k(X,U)k(U,U)− 1k(U,X)] − 1 3

• message 3 :=w¯ =σ − 2(σ − 2 c S Φ c Φ⊤c +λ − 2I)− 1 c S (Φ cy c): This alone poses
f k (X) − f k (X′)= k k((x x 1 2 . . . , , x x 1 1)) · · · · · .. · · k k ((x x 1 2 , , . . . x x n n − − 1 1)) k k ((x x 1
k k (x (x n − n , 1 x ,x 1) 1) · · · · · · k k (x (x n − n , 1 x ,x n − n − 1) 1) k k (x (x n − n , 1 x ,x n n))
k k ((x x 1 2 . .
, , x x 1 1)) · · · · · .. · · k k ((x x 1 2 , , . .
x x n n − − 1 1)) 0 0 . .
k(xn −0 1 ,x1) ··· k(xn − 1 0 ,xn − 1) 0 0
= . .
... . .
k(xn 0 ,x1) · · · · · · k(xn , 0 xn − 1) k k (x (x n − n , 1 x ,x n n))
• In phase 1, L 2 sensitivity of the hyperpameters is the L 2 gradient clipping constant
We run each random seed of each dataset on 1 CPU and 1 NVIDIA T4 GPU with 16GB
run for 10 different random seeds. RMSE reported in Table 6 shows that random features
world regression experiments in Tables 7 and 8. MCE measures the (estimated) worst differ-
We report MCE in Table 10 and BRI in Table 11.
T v h ar e i o a r b e le m s . A L .1 e t (X H¯o = effdin n i= g' 1 s X in i / eq n u . a I l f i t ∀ y) i . ∈ Su { p 1 p ,

Table 6. Hyperparameters used in the real world regression experiments. For FedLoc L=10ρ to run the Proximal ADMM algorithm #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
Size s 200 2000 2000 5000 5000
FedProx μ 0.5 1.0 0.1 1.0 1.0 1.0 0.1 0.01 0.001 1.0
FedBNR-KD α 10 2 1 0.5 5 2 5 0.5 5 5
Theorem 3.3 Let ω and g have the form of Prop. 3.2. Suppose x,x$'$ Rp satisfies
Theorem 3.5 Let ω and g have the form of Prop. 3.4. Suppose entries of x,x$'$

Table 7. RMSE reported. Generalization setting. FedBNR URK: removing random features from FedBNR; \{} FedBNR: ordinary FedBNR. and denote significantly worse results with p<0.01 and p<0.05 respec- tively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
FedBNR 1.00 0.97 0.24 0.52 3.16 5.33 0.40 0.41 4.41 4.51

Table 8. Maximum calibration error (MCE) reported. Generalization setting. and denote significantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
Central GP 0.03 0.16 0.42 0.16 0.40
FedBNR 0.09 0.32⇑ 0.67⇑ 0.62⇑ 0.62⇑ 0.72⇑ 0.15 0.64⇑ 0.39⇓ 0.31⇓
FedBNR-KD 0.10 0.11 0.33 0.35 0.47 0.48 0.15 0.17 0.49 0.50

Table 9. Brier score (BRI) reported. Generalization setting. and denote significantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
Central GP 0.16 0.20 0.24 0.18 0.24
FedBNR-KD 0.18 0.18 0.23 0.23 0.25 0.26 0.19 0.20 0.25 0.26

Table 10. Maximum calibration error (MCE) reported. Personalization setting. and denote significantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
FedBNR 0.03 0.06 0.40 0.48 0.08 0.10 0.19 0.14 0.41 0.56
FedBNR-KD 0.03 0.05↓ 0.25⇓ 0.30⇓ 0.12⇑ 0.17⇑ 0.19 0.12 0.47⇑ 0.55

Table 11. Brier score (BRI) reported. Personalization setting. and denote significantly worse results with p<0.01 and p<0.05 respectively; and denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP

10 100 10 100 10 100 10 100 10 100
DEC-apx 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.20⇑ 0.17⇓ 0.24 0.27
FedBNR 0.17 0.18 0.24 0.25 0.18 0.19 0.19 0.20 0.24 0.27
FedBNR-KD 0.17 0.18↓ 0.21⇓ 0.22⇓ 0.19⇑ 0.20⇑ 0.19 0.19⇓ 0.25⇑ 0.27
/ 1 0 . 1 0 0 7 / s 1 0 9 9 4 - 0 2 5 - 0 6 9 3 6 - 5 .

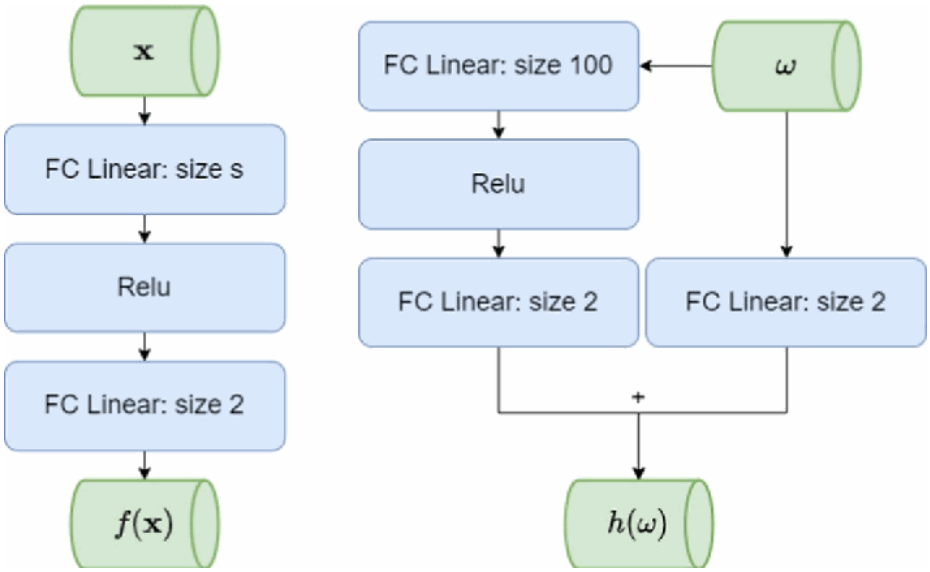

Fig. 6. Architecture of DNNs used in real world regression experiments. Left: the feature extractor for x;

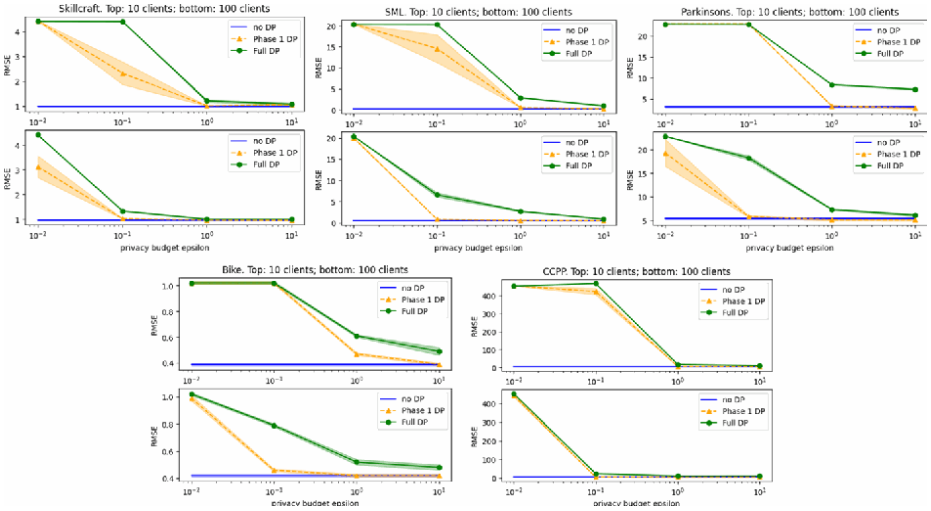

Fig. 7. FedBNR+DP results. Phase 1 DP: add DP protection to model parameters θ only; full DP: add DP

References

1. Machine Learning (2026) 115:72
2. Received: 22 January 2025 / Revised: 14 July 2025 / Accepted: 17 November 2025
3. Machine Learning (2026) 115:72 Page 3 of 36 72
4. Random Fourier features (RFF) (Rahimi & Recht, 2007) can be used to approximate
5. Theorem 2.1 (Bochner's theorem, e.g. (Rudin, 2017)). A continuous stationary kernel
6. Machine Learning (2026) 115:72 Page 5 of 36 72
7. GPs, including generalized product of experts (Cao & Fleet, 2014) and robust Bayesian
8. Works of Hinton and Salakhutdinov (2007), Calandra et al. (2016) either pretrain a deep
9. Dropout (Gal & Ghahramani, 2016) for better calibration. Ober et al. (2021) proposes to use
10. Garnelo et al. (2018) designed a special architecture that makes it possible for DNNs to
11. Bartlett, 2000), DTC (Seeger et al., 2003), FITC (Snelson & Ghahramani, 2005), and PITC
12. Machine Learning (2026) 115:72 Page 7 of 36 72
13. Prop. 3.2. Fenton-Wilkinson approximation (Fenton, 1960) is used to model the summation
14. Machine Learning (2026) 115:72 Page 9 of 36 72
15. Prop. 3.4, with Hoeffding's inequality (Hoeffding, 1994) similarly to Thm. 3.1.
16. Machine Learning (2026) 115:72 Page 11 of 36 72
17. One commonly used method for aggregation is the FedAvg (McMahan et al., 2017) heu-

18. Polychroniadou, 2020; Li et al., 2020).
19. Machine Learning (2026) 115:72 Page 13 of 36 72
20. Theorem 3.6 (e.g. (Kerkouche et al., 2021)). A Gaussian Mechanism is (ϵ,δ)-DP with
21. Machine Learning (2026) 115:72 Page 15 of 36 72
22. FL algorithm that has a global model; (2) FedProx (Li et al., 2020), a non-Bayesian FL
23. Machine Learning (2026) 115:72 Page 17 of 36 72
24. Machine Learning (2026) 115:72 Page 19 of 36 72
25. FedAvg (2017) N/A N/A N/A
26. FITC (2005) U, k(Xc,U) k(X,U)
27. Problematic term: terms Distributed GP (2015) x$\star$/$\Phi\star$ or k(x$\star$,Xc)Xc,yc
28. Distr. Variational GPs (2014; U k(U,) Xc
29. Distributed GP Deisenroth and Ng (2015) introduces a collection of works that uses the
30. Machine Learning (2026) 115:72 Page 21 of 36 72
31. Machine Learning (2026) 115:72 Page 23 of 36 72
32. Machine Learning (2026) 115:72 Page 25 of 36 72
33. Size s 200 2000 2000 5000 5000
34. Machine Learning (2026) 115:72 Page 27 of 36 72
35. Wilkinson approximation (Fenton, 1960) to model URKm
36. Machine Learning (2026) 115:72 Page 29 of 36 72
37. Machine Learning (2026) 115:72
38. Machine Learning (2026) 115:72 Page 31 of 36 72
39. Achituve, I., Shamsian, A., Navon, A., Chechik, G., & Fetaya, E. (2021). Personalized federated learning
40. Begoli, E., Bhattacharya, T., & Kusnezov, D. (2019). The need for uncertainty quantification in machine-
41. Boenisch, F., Dziedzic, A., Schuster, R., Shamsabadi, A. S., Shumailov, I., & Papernot, N. (2021). When the
42. Byrd, D., & Polychroniadou, A. (2020). Differentially private secure multi-party computation for federated
43. Calandra, R., Peters, J., Rasmussen, C. E., & Deisenroth, M. P. (2016). Manifold Gaussian processes for
44. Cao, Y., & Fleet, D. J. (2014). Generalized product of experts for automatic and principled fusion of Gauss-
45. Cuturi, M. (2009). Positive definite kernels in machine learning. arXiv preprint arXiv:0911.5367.
46. Damianou, A., & Lawrence, N.D. (2013). Deep gaussian processes. In: Artificial intelligence and statistics,
47. Danquah, B., Riedmaier, S., Rühm, J., Kalt, S., & Lienkamp, M. (2020). Statistical model verification and
48. Deisenroth, M., & Ng, J.W. (2015). Distributed gaussian processes. In: International conference on machine
49. Machine Learning (2026) 115:72 Page 33 of 36 72
50. Fenton, L. (1960). The sum of log-normal probability distributions in scatter transmission systems. IRE
51. Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in
52. Gal, Y., Van Der Wilk, M., & Rasmussen, C. E. (2014). Distributed variational inference in sparse Gaussian

53. Roscher, R., Shahzad, M., Yang, W., Bamler, R., & Zhu, X. X. (2022). A survey of uncertainty in deep
54. He, B., & Ozay, M. (2022). Feature kernel distillation. In: International conference on learning representations.
55. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In: Proceedings
56. Hensman, J., Fusi, N., & Lawrence, N. D. (2013). Gaussian processes for big data. arXiv preprint
57. Hinton, G. E., & Salakhutdinov, R. R. (2007). Using deep belief nets to learn covariance kernels for Gaussian
58. Hoang, T. N., Hoang, Q. M., & Low, B. K. H. (2016). A distributed variational inference framework for
59. Hoeffding, W. (1994). Probability inequalities for sums of bounded random variables. The collected works
60. Iyer, A., Zhang, Y., Prasad, A., Tao, S., Wang, Y., Schadler, L., Brinson, L. C., & Chen, W. (2019). Data-
61. Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., & Suresh, A. T. (2020). Scaffold: Stochas-
62. Kerkouche, R., Ács, G., Castelluccia, C., & Genevès, P. (2021). Privacy-preserving and bandwidth-efficient
63. Konečny, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., & Bacon, D. (2016). Federated learn-
64. Kontoudis, G. P., & Stilwell, D. J. (2023). Decentralized federated learning using gaussian processes. In:
65. Kontoudis, G. P., & Stilwell, D. J. (2024). Scalable, federated gaussian process training for decentralized
66. Kristiadi, A., Hein, M., & Hennig, P. (2020). Being Bayesian, even just a bit, fixes overconfidence in ReLU
67. Lederer, A., Umlauft, J., & Hirche, S. (2019). Uniform error bounds for gaussian process regression with
68. Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in
69. Li, Y., Zhou, Y., Jolfaei, A., Yu, D., Xu, G., & Zheng, X. (2020). Privacy-preserving federated learning frame-
70. Liu, H., Ong, Y.-S., Shen, X., & Cai, J. (2020). When Gaussian process meets big data: A review of scalable
71. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. (2017). Communication-efficient learn-
72. Mohri, M., Sivek, G., & Suresh, A. T. (2019). Agnostic federated learning. In: International conference on
73. Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). A survey
74. Nickson, T., Gunter, T., Lloyd, C., Osborne, M. A., & Roberts, S. (2015). Blitzkriging: Kronecker-structured
75. Ober, S. W., Rasmussen, C. E., & Wilk, M. (2021). The promises and pitfalls of deep kernel learning. In:
76. Oliva, J. B., Dubey, A., Wilson, A. G., Póczos, B., Schneider, J., & Xing, E. P. (2016). Bayesian nonparamet-
77. Quinonero-Candela, J., & Rasmussen, C. E. (2005). A unifying view of sparse approximate Gaussian process

78. Rahimi, A., & Recht, B. (2007). Random features for large-scale kernel machines. Advances in Neural Infor-
79. Reisizadeh, A., Mokhtari, A., Hassani, H., Jadbabaie, A., & Pedarsani, R. (2020). Fedpaq: A communication-
80. Riedmaier, S., Danquah, B., Schick, B., & Diermeyer, F. (2021). Unified framework and survey for model
81. Riedmaier, S., Ponn, T., Ludwig, D., Schick, B., & Diermeyer, F. (2020). Survey on scenario-based safety
82. Rudin, W. (2017). Fourier analysis on groups. Garden City: Courier Dover Publications.
83. Saatçi, Y. (2012). Scalable inference for structured Gaussian process models. PhD thesis, Citeseer.
84. Sattler, F., Wiedemann, S., Müller, K.-R., & Samek, W. (2019). Robust and communication-efficient feder-
85. Schwaighofer, A., & Tresp, V. (2002). Transductive and inductive methods for approximate Gaussian process
86. Seeger, M. W., Williams, C. K., & Lawrence, N. D. (2003). Fast forward selection to speed up sparse Gauss-
87. Shoham, N., Avidor, T., Keren, A., Israel, N., Benditkis, D., Mor-Yosef, L., & Zeitak, I. (2019). Overcoming
88. Sinha, A., & Duchi, J. C. (2016). Learning kernels with random features. Advances in Neural Information
89. Smith, M. T., Álvarez, M. A., & Lawrence, N. D. (2021). Differentially private regression and classification
90. Smola, A., & Bartlett, P. (2000). Sparse greedy Gaussian process regression. Advances in Neural Information
91. Snelson, E., & Ghahramani, Z. (2005). Sparse Gaussian processes using pseudo-inputs. Advances in Neural
92. Tang, M., Ning, X., Wang, Y., Wang, Y., & Chen, Y. (2021). Fedgp: Correlation-based active client selection
93. Thompson, J. J., Blair, M. R., Chen, L., & Henrey, A. J. (2013). Video game telemetry as a critical tool in the
94. Titsias, M. (2009). Variational learning of inducing variables in sparse Gaussian processes. In: Artificial intel-
95. Tran, G.-L., Bonilla, E. V., Cunningham, J., Michiardi, P., & Filippone, M. (2019). Calibrating deep convolu-
96. Triastcyn, A., & Faltings, B. (2019). Federated learning with bayesian differential privacy. In: 2019 IEEE
97. Truex, S., Baracaldo, N., Anwar, A., Steinke, T., Ludwig, H., Zhang, R., & Zhou, Y. (2019). A hybrid approach
98. Tsanas, A., Little, M., McSharry, P., & Ramig, L. (2009). Accurate telemonitoring of Parkinson's disease
99. Tüfekci, P. (2014). Prediction of full load electrical power output of a base load operated combined cycle
100. Machine Learning (2026) 115:72 Page 35 of 36 72
101. Ve, S., & Cho, Y. (2020). A rule-based model for Seoul bike sharing demand prediction using weather data.
102. Verrelst, J., Rivera, J. P., Moreno, J., & Camps-Valls, G. (2013). Gaussian processes uncertainty estimates in
103. Wei, K., Li, J., Ding, M., Ma, C., Yang, H. H., Farokhi, F., Jin, S., Quek, T. Q., & Poor, H. V. (2020). Feder-

104. Wilcoxon, F. (1992). Individual comparisons by ranking methods. In: Breakthroughs in statistics, pp. 196–
105. Wilson, A. G., Dann, C., & Nickisch, H. (2015). Thoughts on massively scalable Gaussian processes. arXiv
106. Wilson, A., & Nickisch, H. (2015). Kernel interpolation for scalable structured Gaussian processes (kiss-gp).
107. Xia, X., Sivonxay, E., Helms, B. A., Blau, S. M., & Chan, E. M. (2023). Accelerating the design of multishell
108. Xie, A., Yin, F., Xu, Y., Ai, B., Chen, T., & Cui, S. (2019). Distributed Gaussian processes hyperparameter
109. Yin, F., Lin, Z., Kong, Q., Xu, Y., Li, D., Theodoridis, S., & Cui, S. R. (2020). Fedloc: Federated learning
110. Yurochkin, M., Agarwal, M., Ghosh, S., Greenewald, K., & Hoang, N. (2019). Statistical model aggregation
111. Zamora-Martinez, F., Romeu, P., Botella-Rocamora, P., & Pardo, J. (2014). Online learning of indoor tem-
112. Zhu, H., Xu, J., Liu, S., & Jin, Y. (2021). Federated learning on non-iid data: A survey. Neurocomputing,